

Engineering Certified JSON Derivers for Rocq with ELPI

Will Thomas

University of Kansas

Abstract. Rocq developments often need executable serializers for data exchange, testing, and extracted applications, but hand-written serializers are tedious and easy to disconnect from their intended inverse. We present an ELPI-based deriver for `rocq-json` that automatically generates `Jsonifiable` instances for a practical fragment of Rocq inductive and record types. Each generated instance contains an encoder, a decoder, and a proof that decoding the encoded value succeeds with the original value. The implementation is not intended as a new theory of generic programming, nor does the underlying JSON library aim to be an RFC-complete JSON implementation. Instead, the contribution is a performance-conscious and extraction-friendly engineering pattern for mapping Rocq data to a JSON-like interchange format while preserving an explicit invertibility theorem. The accompanying artifact rebuilds the tool, probes Corelib and Stdlib declarations, classifies unsupported cases, compares generated and handwritten extracted code on a large enumeration benchmark, and regenerates the result tables used by this paper.

1 Introduction

Serialization code sits at an awkward boundary in proof assistant projects. It is usually mundane enough that users do not want to write it by hand, but it is important enough that a bug can invalidate testing infrastructure or extracted tools. In Rocq, this tension is especially visible for data types that are defined primarily for proofs and then reused in extracted programs.

`rocq-json` addresses this problem with a typeclass:

```
to_JSON : A -> JSON,  from_JSON : JSON -> Result A string
```

together with the law

```
forall a, from_JSON (to_JSON a) = res a.
```

The deriver described here generates all three components for many ordinary inductive and record types. It is implemented in `coq-elpi` and exposed through the command `Elpi derive.jsonifiable T`.

This is a tool paper about useful engineering rather than a claim of novel foundations. The interesting part is the integration: ELPI inspection of Rocq

declarations, generated code that remains suitable for extraction, typeclass parameters for nested serializable fields, and automatic proof construction for the round-trip theorem. The same ingredients are broadly useful for other derivers that construct computational content and accompanying correctness proofs.

2 Tool Overview

The user-facing interface is deliberately small. Given a declaration such as an enumeration, algebraic data type, parameterized container, or record, the user invokes the deriver and receives a registered typeclass instance. Nullary constructors are encoded as JSON strings. Constructors with arguments are encoded as singleton JSON objects whose key is the constructor name and whose value stores named fields. Records are encoded as JSON objects keyed by projection names.

For parameterized data types, the generated instance abstracts over `Jsonifiable` instances for the type parameters. For recursive types, the generated encoder and decoder use structurally recursive definitions. The current implementation also supports a conservative collection of nested recursive occurrences through standard data containers such as lists, options, and products. More general nested or mutually recursive types are reported as unsupported rather than handled partially by ad hoc guesses.

The generated proof is part of the public contract. A successful derivation does not merely produce functions with plausible behavior; it registers a `canonical_jsonification` proof. This property is intentionally one-sided. It says that values produced by the encoder can be decoded back to the original value. It does not claim that every accepted JSON value is in a canonical printed form.

3 Implementation

The deriver follows the shape of the Rocq declaration. ELPI first reads the inductive or record view, classifies parameters and constructor arguments, and then builds separate skeletons for the encoder, decoder, proof, and final typeclass instance. The skeletons are elaborated by Rocq before being added as constants. This staging has been useful in practice: ELPI performs structural analysis and code generation, while Rocq remains responsible for checking the terms that become part of the trusted development.

Two engineering choices are central. First, field serializers are kept abstract through typeclass arguments where possible. This makes generated instances compose with existing hand-written or derived instances, including standard containers. Second, decoder generation is performance-sensitive. Large enumerations are decoded using a string decision tree that extracts to direct pattern matching over characters, avoiding a long chain of string equality tests. The resulting extracted code is close to the shape a careful user would write by hand.

The proof-generation strategy is mixed. General recursive and record cases use Rocq proof automation specialized to the generated encoders and decoders. Pure enumerations use a direct generated proof term, avoiding proof-search overhead on a common stress case. This division is not theoretically deep, but it matters for predictable tool latency.

4 Evaluation

We evaluate two questions: how broadly the deriver applies to existing Rocq declarations, and whether generated extracted code is competitive with hand-written code on a stress case.

4.1 Applicability

The artifact includes a benchmark script that scans installed Corelib and Stdlib sources for inductive-like declarations, probes each candidate in isolation, records diagnostics, and then compiles a single benchmark file containing all successful derivations. This workflow is deliberately conservative: unsupported declarations remain visible as categorized failures.

Table 1. Corelib/Stdlib applicability summary.

Metric	Count
Discovered declarations	343
Probed declarations	343
Successful probes	92
Probe failures	251
Probe timeouts	0
Timed successful derivations	91

Table 2. Successful derivations by declaration shape and library.

Shape	Count	Library	Count
Inductive	58	Corelib	51
Variant	19	Stdlib	40
Record/Structure	12		

The dominant failure categories are expected limitations of the current data fragment: propositions, indexed families, records containing proof or function fields, coinductive declarations, universe-polymorphic cases, and declarations

Table 3. Failure categories reported by the probe benchmark.

Category	Count
prop-target-not-supported	154
to-json-generation-not-supported	35
record-generation-not-supported	32
scanner-or-logical-path	18
to-json-elaboration-failed	7
indexed-family-not-supported	3
coinductive-not-supported	1
universe-polymorphism-not-supported	1

found by source scanning but not addressable through the generated logical path. These failures are useful artifact outputs, not discarded noise, because they state where the deriver should not currently be used.

4.2 Derivation Time

The combined successful benchmark records Rocq’s **Time** output for each derived instance. In the generated run used for this draft, the combined benchmark compiled successfully with wall-clock time 5.86 seconds; the sum of Rocq-reported derivation transactions was 5.09 seconds. The mean derivation time was 0.056 seconds, the median was 0.031 seconds, the 90th percentile was 0.126 seconds, and the maximum was 0.500 seconds.

Table 4. Slowest successful derivations in the broad benchmark.

Declaration	Kind	Seconds
Corelib.Init.Byte.byte	Inductive	0.500
Stdlib.btauto.Algebra.poly	Inductive	0.267
Corelib.Init.Hexadecimal.uint	Inductive	0.255
Stdlib.micromega.RingMicromega.Op1	Inductive	0.225
Stdlib.rtauto.Rtauto.form	Inductive	0.221
Stdlib.rtauto.Rtauto.proof	Inductive	0.190
Stdlib.setoid_ring.Ring_polynom.Mon	Inductive	0.137
Stdlib.micromega.VarMap.t	Inductive	0.136

4.3 Extracted Code Performance

The extraction benchmark compares the generated **Jsonifiable** instance for an enumeration with 256 constructors against a handwritten instance with the same representation. Each run serializes and deserializes every constructor. The benchmark is intentionally small but targeted: earlier versions of the generated

decoder used equality chains whose extracted runtime was visibly worse than handwritten pattern matching.

Table 5. Extracted `enum_256` round-trip benchmark.

Implementation	Runs	Mean seconds	Median seconds
Generated	10	0.000013	0.000013
Handwritten	10	0.000014	0.000013

The intended conclusion is not that this microbenchmark predicts every serializer workload. Rather, it checks that the deriver does not impose an obvious extraction penalty on a case that stresses constructor dispatch.

5 Artifact

The artifact is part of the contribution. Tool papers benefit from evaluation that reviewers can rerun without reconstructing informal commands from prose. Our artifact provides a Docker path and a native path. The main script rebuilds the project from a clean state, runs the broad applicability benchmark, runs the extraction benchmark, and regenerates the LaTeX macros included in this paper.

The generated files are meant to be read directly. The JSON summary gives the aggregate result, the CSV files contain row-level data, and the Markdown failure report explains each failure category with representative Rocq diagnostics. This setup makes the boundary of the tool explicit: reviewers can see both what works and what is outside the supported fragment.

6 Limitations

The current deriver targets computational data in `Type` or `Set`. It does not attempt to serialize arbitrary propositions, dependent families, coinductive values, function fields, or records whose field types depend on earlier fields. Some nested recursion through standard containers is supported, but general nested and mutually recursive types remain outside the current design. This is a deliberate usability choice: partial support for a few accidental nested forms can be worse than a clear diagnostic when users cannot predict which structurally similar declarations will work.

The JSON representation is also intentionally modest. The paper evaluates the derivation method and the certified round-trip property, not complete adherence to the JSON RFC or interoperability with every external JSON parser.

7 Conclusion

`rocq-json`'s ELPI deriver demonstrates that useful certified serialization support can be built with a small user-facing interface and an honest artifact. The generated instances combine executable encoders and decoders with Rocq-checked invertibility proofs, apply across a meaningful fragment of existing library declarations, and extract to competitive code on a large-enumeration stress test. The engineering is specific to JSON, but the pattern is broader: use ELPI to inspect declarations, generate computational content and proof obligations together, and evaluate both applicability and runtime behavior as first-class artifact outputs.